GPAS: OPTIMIZER-AWARE ALLOCATION FOR MULTI-TEACHER ON-POLICY DISTILLATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Multi-teacher on-policy distillation (MOPD) combines task-specific teacher feedback in a shared student. Its training budget must accommodate tasks whose responses differ in length and whose gradients vary in reliability. Dense top-k supervision already uses several teacher scores at each position, but different prompts and student-generated trajectories still produce different updates. We introduce Gradient-Preconditioned Adaptive Sampling (GPAS), a micro-batch allocation method that uses this remaining variation to decide how much computation each task receives. GPAS averages each task's gradients under fixed objective weights and assigns more micro-batches to tasks with larger fluctuations after the optimizer's coordinate scaling. It reuses gradients from ordinary training and supports a cost-aware mode that also accounts for task-dependent running time. We study four-domain distillation into Qwen3-1.7B, comparing training efficiency, capability integration, and allocation dynamics under existing dense distillation losses. **[TODO: Report the measured efficiency difference, per-domain capability changes, and the contribution of optimizer-aware allocation.]**

1 INTRODUCTION

Multi-teacher on-policy distillation (MOPD) provides a practical way to combine capabilities in one language model. The student generates responses, and each response is scored by the teacher assigned to its task (Ma et al., 2026). This allows specialist teachers to be developed separately and their feedback to be combined during student training. The same approach can add a new capability while using an earlier model as a teacher to preserve existing ones (Liu et al., 2026). Once the teachers are available, a recurring engineering question is how to divide the student's training budget among their tasks.

Existing recipes address several parts of this problem. Open-MOPD adjusts loss weights to compensate for response-length imbalance and to emphasize domains with larger teacher–student gaps (Gao et al., 2026). D^3-MOPD changes the sampling mixture using the remaining gap and its recent rate of decrease (Sun et al., 2026). These approaches make the allocation responsive to task balance and learning progress. They also motivate a complementary question: for a chosen balance of task objectives, how many samples are needed to obtain a reliable update from each task?

Consider two equally weighted tasks. One gives similar gradients across micro-batches, while the other gives gradients that fluctuate widely with the sampled prompts and responses. Equal counts spend the same computation on both estimates even though the second is less precise. Giving the second task more micro-batches can make the combined update more reliable. Its objective weight can stay the same: first average the gradients within each task, then combine those averages with the chosen weights. Task importance and the amount of computation used to estimate its update are separate choices.

This distinction remains useful with dense distillation. MOPD already supports a teacher top-k loss, and recent methods improve how retained tokens and the distribution tail contribute to supervision (Ma et al., 2026; Huang & Wei, 2026). Even when all retained-token contributions are computed at a fixed prefix, different prompts and student rollouts still yield different gradients. The relevant signal for allocation is therefore the variability of the actual training gradient. Teacher loss alone does not describe that variability: two tasks can have similar losses but different gradient directions and response lengths.

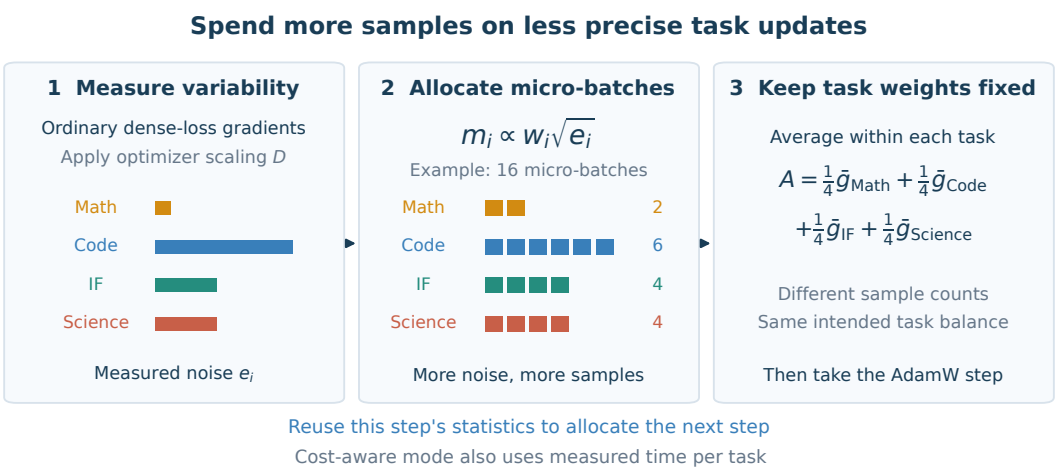

Figure 1: GPAS separates task importance from computation. Task means are combined with fixed weights w, while counts m adapt to the variation of ordinary dense-distillation gradients. The optimizer's pre-step scaling D determines how those fluctuations affect the update. Observed noise and running time guide the next allocation. The task profiles are schematic.

We propose Gradient-Preconditioned Adaptive Sampling (GPAS), which allocates micro-batches using the gradient variation observed during ordinary MOPD training. GPAS measures this variation after the optimizer's coordinate scaling. This matters because a fluctuation has a larger effect on an update when the optimizer amplifies the coordinates in which it occurs. With fixed task weights, the resulting rule gives each task a count proportional to its weighted preconditioned standard deviation. A cost-aware mode also uses measured running time, allowing the same scheduler to target efficiency per step or per GPU hour.

Our study follows this practical sequence: compare learning curves and per-domain capability, examine where GPAS spends computation, and measure the contributions of optimizer scaling and running time. All core allocation comparisons use the same existing top-k loss. A second dense loss tests the same scheduling idea with tail-aware supervision. **[TODO: Summarize the main efficiency and capability results and the allocation pattern that explains them.]**

The paper makes three contributions:

- a practical separation of task weights and micro-batch counts for MOPD, so computation can adapt while each task retains its intended contribution to the expected gradient;
- an optimizer-aware allocation method that reuses micro-batch gradients, with a cost-aware mode based on measured task time;
- a four-domain study relating training efficiency and capability integration to gradient variability, learning progress, and the allocation of computation.

2 Distillation and Task Allocation

2.1 Dense Supervision on Student Rollouts

There are M tasks with prompt distributions $\mathcal{D}_i$ and assigned frozen teachers π_{T_i}. The student and teachers share a vocabulary. For a task-i prompt x, the student generates a response y. At position t, the teacher scores the student-generated prefix $c = (x, y_{<t})$. Write $p_\theta(a \mid c) = \pi_\theta(a \mid c)$ and $q_i(a \mid c) = \pi_{T_i}(a \mid c)$.

We use the teacher top-k loss from MOPD (Ma et al., 2026). Let $S_i(c) = \mathrm{TopK}_k(q_i(\cdot \mid c))$. The position loss is

$$\ell_i^{(k)}(\theta; c) = \sum_{a \in S_i(c)} \left[p_\theta(a \mid c) \log \frac{p_\theta(a \mid c)}{q_i(a \mid c)} - p_\theta(a \mid c) + q_i(a \mid c) \right]. \tag{1}$$

The probabilities retain their full-vocabulary normalization. The $-p_\theta + q_i$ correction makes the loss smallest when student and teacher probabilities agree on the retained tokens. Both the teacher scores and the generated prefixes are held fixed during the update. This retained-support objective gives a dense gradient at each observed position using a small teacher payload. GPAS operates on the resulting micro-batch gradients; the allocation method also applies to other distillation losses.

We average position losses within each response, then average responses within a task. Positive weights w_i, fixed before training and summing to one, specify the desired balance:

$$F(\theta) = \sum_{i=1}^{M} w_i L_i(\theta). \tag{2}$$

Here L_i is the expected response-mean loss under the current rollout distribution. Each update differentiates the loss on those sampled prefixes; Appendix A.4 gives the corresponding step-local objective. Our main study uses equal task weights.

2.2 Averaging task gradients before combining them

A micro-batch contains b prompts from one task. An optimizer step processes G micro-batches, with task i receiving m_i of them. The counts sum to G and satisfy $m_{\min} \le m_i \le m_{\max}$. We use $m_{\min} \ge 2$ to estimate each task's variability from the same step.

Let $g_{i,s}$ be the gradient of the response-mean loss in micro-batch s of task i. GPAS accumulates

$$A = \sum_{i=1}^{M} w_i \underbrace{\frac{1}{m_i} \sum_{s=1}^{m_i} g_{i,s}}_{\text{task mean}}. \tag{3}$$

Implementation only requires multiplying each micro-batch loss by w_i/m_i. For example, an equally weighted task contributes one quarter of the combined mean whether it receives two or six micro-batches; six micro-batches give a more precise estimate of that mean.

Counts are chosen before drawing the step's data. If fresh task-i micro-batches have mean μ_i at the current training state, then $\mathbb{E}[A \mid m] = \sum_i w_i \mu_i$. Thus, the scheduler can change counts without changing the intended weighted mean. Response-level averaging also prevents a task from gaining extra loss weight simply because its responses are longer. This addresses the length imbalance highlighted by Open-MOPD (Gao et al., 2026), while making the response the unit of averaging within each task.

3 GPAS: Spending Computation on Noisy Task Estimates

3.1 Measure fluctuations where they affect the update

GPAS asks how much a task's micro-batch gradient varies around its own mean. A large gradient shared by every micro-batch is a consistent learning signal. A large fluctuation between micro-batches is something additional samples can average out. We measure the latter after the optimizer's coordinate scaling, so the statistic reflects the coordinates in which the update will move.

Let D be the diagonal scaling available at the start of the step. For SGD, $D = I$; for Adam-type optimizers, $D = \mathrm{diag}((\sqrt{\widehat{v}_{\mathrm{pre}}} + \epsilon)^{-1})$, where $\widehat{v}_{\mathrm{pre}}$ is the pre-step bias-corrected second moment. Define the task's noise and the combined-gradient variance as

$$e_i = \mathbb{E}\|D(g_i - \mu_i)\|_2^2, \qquad V(m) = \mathbb{E}\|D(A - \mu)\|_2^2 = \sum_{i=1}^{M} \frac{w_i^2 e_i}{m_i}, \tag{4}$$

where $\mu = \sum_i w_i \mu_i$. The equality follows from conditionally independent micro-batches at a fixed training state. It captures a simple benefit of averaging: doubling a task's count halves its contribution to the variance. We use the pre-step D as a local measure for AdamW and leave the optimizer update unchanged.

The minimum-variance continuous allocation is

$$m_i \propto w_i \sqrt{e_i}. \tag{5}$$

This is the classical Neyman allocation (Neyman, 1934; Cochran, 1977), applied to MOPD micro-batches in the optimizer's coordinates. For equally weighted tasks, nine times the noise calls for three times as many micro-batches. The square root reflects diminishing returns: as a task receives more samples, its mean becomes more precise and further samples help less. We scale the counts to sum to G, enforce the bounds, and round by largest remainder. Appendix A.5 gives the derivation and Appendix A.6 connects the variance to a local descent bound.

3.2 Reuse the gradients already computed

After processing a task's micro-batches, let $\bar{g}_i$ be their sample mean. The observed noise is

$$\widehat{e}_i = \frac{1}{m_i - 1} \sum_{s=1}^{m_i} \|D(g_{i,s} - \bar{g}_i)\|_2^2. \tag{6}$$

We smooth these measurements across steps, $\widetilde{e}_i \leftarrow \rho \widetilde{e}_i + (1-\rho)\widehat{e}_i$, and use $\widetilde{e}_i$ in the next allocation. Smoothing lets individual long or unusual responses influence the schedule gradually. The first step uses Uniform counts and initializes the moving averages.

Tasks are processed in blocks. A running mean and a scalar sum of squared deviations give Equation 6 without retaining every micro-batch gradient. The same parameter-sized mean buffer is reused for each task. The additional work consists of vector reductions and the count calculation; rollout, teacher scoring, and backward passes are the ordinary training passes. We measure the memory and running-time cost of collecting these statistics.

This measurement also makes the schedule easy to interpret. We log task loss, response length, measured noise, and assigned counts together. Loss tracks the remaining distillation gap; noise tracks how variable the estimate of its update is. GPAS can therefore give two tasks with similar losses different counts, or change their relative counts as the optimizer's scaling evolves. These trajectories connect the allocation to the training process directly, without requiring a separate token-level noise model.

3.3 Account for the time a micro-batch takes

Tasks also differ in cost. A long code response may occupy the rollout engine and teacher service much longer than a short instruction-following response. GPAS has a cost-aware mode that uses measured time alongside gradient noise.

Let τ_i be the marginal time for a task-i micro-batch and C the fixed time for synchronization, the optimizer, and other step-level work. For a step-time model $T(m) = C + \sum_i m_i \tau_i$, the scheduler minimizes

$$J_C(m) = T(m)V(m) = \left(C + \sum_i m_i \tau_i\right) \sum_i \frac{w_i^2 e_i}{m_i}. \tag{7}$$

This local criterion balances precision per update against updates per unit time. With four tasks, we enumerate the feasible integer count vectors. When fixed time is negligible, the continuous rule becomes $m_i \propto w_i \sqrt{e_i/\tau_i}$: an expensive task needs more variability to justify the same number of micro-batches. As fixed step time grows, the choice approaches the per-step rule. We fit the timing model to the training trace and evaluate efficiency using end-to-end GPU hours.

4 Experimental Setup

4.1 A four-domain distillation setting

We study mathematics, code, instruction following, and science with a Qwen3-1.7B student in non-thinking mode. Mathematics and instruction following use domain-specific Qwen3-1.7B RL teachers. Code and science are routed to the same frozen Qwen3-4B model through separate task endpoints. This setting combines specialized feedback and a larger generalist teacher in one student.

1. Choose counts m from the previous noise and time averages; use Uniform on the first step. Read the pre-step optimizer scaling D.
2. For each task, generate m_i micro-batches and obtain its teacher's top-k scores on the student responses.
3. Compute the ordinary dense loss. Accumulate the task's gradients, record their variation under D, and add the task mean to A with weight w_i.
4. Update the noise and time averages, then apply the usual optimizer step with gradient A.

Algorithm 1: One training step with GPAS.

The default loss is Equation 1 with teacher $k = 64$, and each task has weight $w_i = 1/4$. Each step processes $G = 16$ micro-batches of $b = 4$ responses, with $2 \leq m_i \leq 8$. Uniform uses four micro-batches per task. Runs last 500 steps; each task supplies a shuffled stream of 16,000 unique prompts, and each response is capped at 4,096 tokens. There is one optimizer update per fresh rollout batch. Noise and time estimates use EMA decay 0.9. All runs use AdamW, the same task prompt orders, and one training seed per method.

One 96-GB GPU trains the student, and one 48-GB GPU serves student rollouts and teacher scoring. The two generalist task endpoints share their teacher weights in memory. Appendix A.1 records the configuration. **[TODO: Complete the exact checkpoint and data revisions, optimizer settings, software versions, and measured resource use.]**

4.2 Comparisons

The core comparison keeps the loss, response averaging, and task weights fixed, varying how the 16 micro-batches are assigned:

- **Uniform**: four micro-batches per task.
- **GPAS (per step)**: counts from smoothed preconditioned gradient noise.
- **GPAS (cost-aware)**: counts from the time–variance criterion in Equation 7.
- **Unpreconditioned allocation**: the same variance rule with $D = I$ for count selection; AdamW still performs the update.
- **D^3 signal, fixed weights**: counts from D^3-MOPD's remaining-gap and descent-velocity signal, combined with our fixed task-mean weights. This compares learning progress with gradient variability as a scheduling signal.

We also include two recipe-level comparisons. **D^3-MOPD scheduling** uses the same gap–velocity scheduling rule but averages responses across the step, so a task's effective weight follows m_i/G. **Open-MOPD** ($K = 1$) uses its student top-16 dense loss, token-share balancing, and gap-aware weighting with one update per rollout batch. Student reward refresh has no intervening update to correct in this $K = 1$ setting. These are adaptations to our model, data, and training budget; their task objectives differ from the fixed-weight comparison. Appendix A.3 specifies the adaptations.

Finally, **TA-OPD + Uniform** and **TA-OPD + GPAS** use the tail-aware loss of Huang & Wei (2026) at the same $k = 64$. This pair evaluates whether the allocation remains useful when the dense supervision also accounts for the omitted probability mass. It brings the study to nine training configurations.

4.3 Capability, efficiency, and allocation traces

Capability and learning curves. We report MATH-500 greedy pass@1, a fixed LiveCodeBench pass@1 slice, IFBench strict accuracy, and GPQA-Diamond average@4. The initial student and assigned teachers are evaluated with the same benchmark protocols. The main table reports all four scores and their unweighted mean. Every 50 steps, we also evaluate 128 fixed held-out prompts per task using the common teacher top-64 loss and equal task weights. This provides a common distillation metric even for recipes trained with different losses or task weights. Shared evaluation seeds allow paired prompt-level bootstrap intervals; these describe evaluation uncertainty, not

Table 1: Capability and training efficiency in the four-domain study. F_{eval} is the common equal-weight teacher top-64 evaluation loss. Time to target is reported within the default-loss, fixed-weight group. Dashes denote inapplicable entries; blank cells await measurements.

Method	Math	Code	IF	Science	Mean	F_{eval}	GPU h to target
Initial student							—
Assigned teachers						—	—
Uniform							
GPAS (per step)							
GPAS (cost-aware)							
Unpreconditioned allocation							
D^3 signal, fixed weights							
D^3-MOPD scheduling							—
Open-MOPD ($K = 1$)							—
TA-OPD + Uniform							—
TA-OPD + GPAS							—

[TODO: Plot common held-out teacher loss against optimizer steps and measured GPU hours. Include Uniform, the two GPAS modes, and the scheduling comparisons; give the recipe-level comparisons in a companion panel.]

Figure 2: Learning progress per update and per unit of computation.

training-seed variation. **[TODO: Insert benchmark versions, the code evaluation date range, and complete decoding settings.]**

Measured efficiency. We plot the common held-out loss against both optimizer steps and GPU hours. GPU hours include all time on the two reserved devices, including waiting, synchronization, statistics, and checkpointing. For the fixed-weight default-loss methods, we also report the time to Uniform's final held-out loss, interpolating between adjacent evaluations and marking unreached targets. Capability scores accompany this loss-based comparison because teacher agreement and task performance need not improve together.

Understanding the allocation. The ordinary training loop logs task loss, its recent rate of change, response length, preconditioned and unpreconditioned noise, assigned counts, and time per micro-batch. These traces show when a task receives more computation and whether the decision follows remaining gap, gradient variability, or cost. We report the time and memory used by the GPAS statistics as part of the implementation cost.

5 Results

5.1 Training efficiency and per-domain capability

Table 1 and Figure 2 evaluate whether the allocation translates into useful training progress. **[TODO: Report measured efficiency and every per-domain score change relative to Uniform. Describe the observed tradeoff between the two GPAS modes and the recipe-level comparisons.]**

[TODO: For each task, align smoothed gradient noise, allocated micro-batches, teacher loss and its rate of change, and response length over training. Show preconditioned and unpreconditioned noise with clear, separate scales.]

Figure 3: Allocation dynamics from the ordinary training trace. The panels connect changes in computation to changes in the task-gradient estimates and learning progress.

5.2 Where does the computation go?

Figure 3 follows the decisions behind the learning curves. The comparison with unpreconditioned allocation isolates the optimizer's coordinate scaling; the fixed-weight D^3 signal compares noise with remaining gap and learning velocity. **[TODO: Describe the main allocation shifts and their timing. Explain where the signals agree or disagree using the task traces, and relate those decisions to the capability and efficiency results.]**

5.3 Cost-aware allocation and reuse with a second dense loss

The cost-aware run uses the observed time of each task alongside its gradient variability. We report the fitted step-time components, observed step times, and the time spent collecting GPAS statistics. **[TODO: Report the measured cost differences and the resulting allocation and GPU-hour differences between the two GPAS modes.]**

The TA-OPD pair reuses the same scheduler with tail-aware supervision. **[TODO: Report the capability and learning-curve differences between TA-OPD + Uniform and TA-OPD + GPAS. Compare the resulting allocation patterns with those under the default teacher top-64 loss.]**

6 Related Work

Multi-teacher distillation in practice. MOPD combines domain teachers through supervision on student-generated responses (Ma et al., 2026); recent systems also use a specialist together with an earlier checkpoint to add skills while preserving general capability (Liu et al., 2026). Open-MOPD identifies response-token imbalance, uneven learning progress, and stale student-dependent rewards, addressing them through loss weighting and reward refresh (Gao et al., 2026). D^3-MOPD adapts domain sampling using the remaining teacher gap and its descent velocity (Sun et al., 2026). GPAS addresses the precision of task-gradient estimates under specified objective weights, using their measured variability and computational cost.

Dense losses and token estimators. MOPD includes both sampled-token policy gradients and a corrected reverse-KL loss on the teacher's top-k tokens (Ma et al., 2026). Open-MOPD uses student top-k candidates with probability-weighted log-ratio advantages; its released dense implementation computes candidate-wise losses before summing across the retained set (Gao et al., 2026; BytedTsinghua-SIA, 2026). EMA-PG combines an exact retained-set contribution with a sampled tail (Zhang & Ba, 2026), TA-OPD includes the probability mass outside the teacher's top-k set (Huang & Wei, 2026), and vOPD uses a control-variate baseline for sampled-token distillation (Oh et al., 2026). We build on existing dense losses and apply GPAS to the micro-batch gradients they produce.

Learning dynamics and capability integration. Teacher–student compatibility and the capabilities supplied by the teacher shape OPD outcomes (Li et al., 2026). CaMOPD studies counteracting

recovery and preservation updates and combines alternating training with gap-based selection (Chen et al., 2026). Recent work also finds that diverse queries can provide broad state coverage while the student continues to absorb supervision slowly (Fu et al., 2026). These studies motivate examining the update process alongside teacher loss. Our measurements relate gradient variability, sample allocation, and learning efficiency.

Task weighting and stochastic optimization. Multi-task learning adapts loss weights and gradient directions (Chen et al., 2018; Sener & Koltun, 2019; Yu et al., 2020); language-model training also adapts data mixtures from learning progress (Albalak et al., 2023; Wang et al., 2025). Importance sampling and online variance reduction improve stochastic gradient estimates (Katharopoulos & Fleuret, 2018; Borsos et al., 2018). GPAS uses within-task standard deviations, following Neyman allocation and its unequal-cost extension (Neyman, 1934; Cochran, 1977). The interaction between non-uniform sampling and adaptive optimizers (Lahire, 2023) motivates measuring those deviations after the optimizer's coordinate scaling.

7 Discussion and Conclusion

GPAS treats task importance and gradient-estimation effort as separate decisions. Existing dense distillation provides the learning signal; the scheduler uses its observed variability to decide where more micro-batches are useful. The same principle gives a per-step rule and a cost-aware rule, both using statistics collected during ordinary training. **[TODO: Conclude with the measured efficiency difference and the task-level learning pattern that explains it.]**

The present study covers a Qwen3-1.7B student, four domains, one training seed per method, and one hardware layout. The method is most directly applicable when an optimizer step contains several micro-batches from each task; larger task collections call for less frequent task measurements. The pre-step optimizer scaling and measured time model provide local allocation signals, and their practical effect is evaluated with the complete AdamW training loop. Task compatibility and student capacity remain important for capability integration. GPAS offers a way to improve how a chosen collection of task objectives uses computation.

AI use statement

Generative AI tools assisted with language editing and manuscript organization. The authors remain responsible for reviewing the text, claims, proofs, code, and experimental results before submission.

Ethics statement

The study evaluates optimization methods for existing language models with reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect capability scores. The final artifact will record data provenance, invalid outputs, and verifier failures to support careful interpretation.

Reproducibility statement

Section 4 specifies the micro-batch composition, fixed objective weights, integer allocation, resource accounting, baselines, held-out evaluation, and matched prompt streams. The current artifact includes the manuscript sources and figure code; earlier synthetic calculations are retained separately from the planned MOPD study. **[TODO: Before submission, add the frozen run configuration, model and tokenizer revisions, fixed task weights, prompt streams, optimizer and allocation states, micro-batch counts, noise and timing logs, per-prompt evaluation outputs, and source data for every empirical figure and table.]**

References

Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. Efficient online data mixing for language model pre-training, 2023. URL https://arxiv.org/abs/2312.02406.

Zalan Borsos, Andreas Krause, and Kfir Y. Levy. Online variance reduction for stochastic optimization. In *Proceedings of the 31st Conference on Learning Theory*, pp. 324–357, 2018. URL https://proceedings.mlr.press/v75/borsos18a.html.

BytedTsinghua-SIA. Open-MOPD: Dense distillation implementation. GitHub, 2026. URL https://github.com/BytedTsinghua-SIA/Open-MOPD. Revision 4809a96cf85a, actor and dense-loss implementation.

Tianlei Chen, Jiao Ou, Ziyuan Liu, Ruiming Tang, Jian Liang, and Han Li. Counteraction-aware multi-teacher on-policy distillation for general capability recovery with domain preservation, 2026. URL https://arxiv.org/abs/2605.27115.

Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 794–803, 2018. URL https://proceedings.mlr.press/v80/chen18a.html.

William G. Cochran. *Sampling Techniques*. Wiley, 3 edition, 1977.

Zixuan Fu, Bingxiang He, Yuxin Zuo, Haohuan Huang, Jinqian Zhang, Ruhang Xiao, Cheng Qian, Qinyu Luo, Huan ang Gao, Yudong Wang, Zhiyuan Liu, Ning Ding, and Chaojun Xiao. Rethinking on-policy distillation of large language models II: One training example, 2026. URL https://arxiv.org/abs/2609.04172.

Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-MOPD: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL https://arxiv.org/abs/2608.19098.

Huipeng Huang and Hongxin Wei. Tail-aware top-k on-policy distillation, 2026. URL https://arxiv.org/abs/2608.14728.

Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 2525–2534, 2018. URL https://proceedings.mlr.press/v80/katharopoulos18a.html.

Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023: Optimization for Machine Learning, 2023. URL https://opt-ml.org/papers/2023/paper20.pdf.

Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan ang Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe, 2026. URL https://arxiv.org/abs/2604.13016.

Jiang Liu, Sudhanshu Ranjan, Prakamya Mishra, Yonatan Dukler, Gowtham Ramesh, Jialian Wu, Ximeng Sun, Wen Xie, Chaojun Hou, Vikram Appia, Zhenyu Gu, Zicheng Liu, and Emad Barsoum. Instella-MoE technical report, 2026. URL https://arxiv.org/abs/2609.00791.

Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. MOPD: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL https://arxiv.org/abs/2606.30406.

Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4): 558–625, 1934. doi: 10.2307/2342192.

Minjae Oh, Sangjun Song, Gyubin Choi, Yunho Choi, and Yohan Jo. KL for a KL: On-policy distillation with control variate baseline, 2026. URL https://arxiv.org/abs/2605.07865.

Qwen Team. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL https://arxiv.org/abs/1810.04650.

Zechen Sun, Zhiwei Zhang, Fei Zhao, Juntao Li, Mu Chuan, Huayu Deng, Guojian Zhan, Wenliang Chen, Yao Hu, and Min Zhang. D^3-MOPD: Adaptive dynamic domain scheduling for efficient multi-teacher distillation, 2026. URL https://arxiv.org/abs/2608.24987.

Zhenting Wang, Guofeng Cui, Kun Wan, and Wentian Zhao. DUMP: Automated distribution-level curriculum learning for RL-based LLM post-training, 2025. URL https://arxiv.org/abs/2504.09710.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL https://arxiv.org/abs/2001.06782.

Lunjun Zhang and Jimmy Ba. EMA policy gradient: Taming reinforcement learning for LLMs with EMA anchor and top-k KL, 2026. URL https://arxiv.org/abs/2602.04417.

A Implementation and Supporting Derivations

A.1 Configuration

Table A1: Configuration of the four-domain study. Entries marked TODO require the corresponding training or evaluation artifact.

Component	Configuration
Student	Qwen3-1.7B (Qwen Team, 2025), non-thinking mode; **[TODO: frozen model and tokenizer revisions]**
Teachers	Domain-specific Qwen3-1.7B RL checkpoints for math and instruction following; one shared Qwen3-4B weight set for code and science. All use the same vocabulary and tokenizer. **[TODO: exact checkpoint identifiers, RL training recipes, and scores]**
Training data	16,000 unique prompts per task, shuffled once and consumed without replacement. **[TODO: datasets, versions, licenses, filters, and held-out split]**
Default loss	Teacher top-64 corrected reverse KL (Equation 1); full-vocabulary probabilities; position mean within response, response mean within task; $w_i = 1/4$
Rollout	One response per prompt and one update per fresh batch; maximum response length 4,096; truncated responses retained. **[TODO: temperature, EOS handling, and decoding settings]**
Step and budget	$b = 4$, $G = 16$, $2 \leq m_i \leq 8$, 500 steps, 32,000 attempted responses per run, one seed per method
Optimizer	AdamW; **[TODO: learning rate and schedule, betas, epsilon, weight decay, clipping, precision, and memory configuration]**
GPAS	Pre-step bias-corrected Adam second moment for D; noise and time EMA decay 0.9; first step Uniform; bounded proportional rounding for per-step counts and integer enumeration for cost-aware counts
Evaluation	128 held-out prompts per task every 50 steps; MATH-500, LiveCodeBench, IF-Bench, and GPQA-Diamond. **[TODO: versions, evaluation seeds, and decoding protocols]**
System	One 96 GB learner GPU and one 48 GB rollout/teacher GPU; the code and science endpoints share weights. **[TODO: GPU models, inference and training software revisions, peak memory, and GPU hours]**

A.2 Dense loss implementation and online statistics

At each student-generated prefix, the teacher returns the identifiers and full-vocabulary log probabilities of its top-64 tokens. The student computes its full-vocabulary log normalizer and gathers

its probabilities at those identifiers. Autodifferentiation is applied to the whole expression in Equation 1, including the $-p_\theta$ term. The retained probabilities are not renormalized within the top-k set, and the loss does not use a sampled tail correction. The operation is dense over the retained set and uses sparse teacher output.

GPAS collects statistics from unweighted micro-batch gradients. For a task block, initialize its running mean $h = 0$ and scalar $S = 0$. After micro-batch s, form $\delta = g_{i,s} - h$, then update

$$h \leftarrow h + \delta/s, \qquad S \leftarrow S + \frac{s-1}{s}\|D\delta\|_2^2. \tag{8}$$

At the end of the block, $\widehat{e}_i = S/(m_i - 1)$ and $A \leftarrow A + w_i h$. The mean buffer is reused for the next task. The same recurrence with $D = I$ records unpreconditioned noise. Reductions use float32 accumulation. Gradients are unscaled before measurement if mixed-precision loss scaling is active, and clipping is applied to the assembled gradient A.

The optimizer state is fixed during this collection. Before an Adam second moment is available, the first Uniform step uses $D = I$. Subsequent steps use the pre-step bias-corrected second moment. The first observation initializes each noise EMA; later observations use decay 0.9. The scheduler reads the completed step's averages before generating the next step's data.

For per-step allocation, set $a_i = w_i\sqrt{\widetilde{e}_i}$ and find a scale λ such that $\sum_i \mathrm{clip}(\lambda a_i, m_{\min}, m_{\max}) = G$. Floor these counts and assign remaining units in descending order of fractional remainder, respecting the bounds; ties follow the fixed task order. Zero-noise tasks receive the lower bound unless the other tasks reach their upper bounds; if all scores are zero, use Uniform. The cost-aware scheduler evaluates all feasible integer vectors using the smoothed noise and timing estimates.

A.3 Comparison recipes

D^3-MOPD scheduling. We use the remaining-gap and descent-velocity construction of Sun et al. (2026). Let $\bar{L}_i(t)$ be the smoothed task loss and L_i^0 the mean of its first five observations. With $W = 10$ and up to $R = 3$ available windows, the composite signal is

$$\begin{aligned} r_i(t) &= \bar{L}_i(t)/L_i^0, \\ v_i(t) &= \max\left\{0, \frac{1}{R'}\sum_{j=0}^{R'-1}\frac{\bar{L}_i(t-(j+1)W) - \bar{L}_i(t-jW)}{\bar{L}_i(t-(j+1)W)}\right\}, \\ s_i(t) &= r_i(t)v_i(t). \end{aligned} \tag{9}$$

We retain the published max-normalization, softmax temperature 0.5, per-domain probability floor 0.10, and multiplicative batch jitter of amplitude 0.30. The scheduler refreshes every 10 steps, using the paper's history warmup and EMA window of 10. All KL denominators use the published floor of 0.15. The signal is computed from the default dense task loss in our pipeline. The resulting mixture is converted to micro-batch counts with the shared bounds $[2, 8]$. Thus this is a controlled adaptation to our loss scale, micro-batch unit, and budget.

The two D^3 rows use the same scheduling rule. The fixed-weight row assembles $A = \sum_i w_i\bar{g}_i$; the recipe-level row assembles $A = \sum_i (m_i/G)\bar{g}_i$. This makes the effect of task weighting visible alongside the comparison of scheduling signals.

Open-MOPD with one update per rollout. We follow the released student top-16 dense candidate loss, token-share balancing, and gap-aware weighting (Gao et al., 2026; BytedTsinghua-SIA, 2026). The released implementation multiplies candidate-wise detached advantages by candidate-wise student probability ratios and then sums over candidates. Each retained token therefore contributes its own gradient. In our setting, the initial task target is uniform, and each batch contains four micro-batches per task. The recipe's token and gap weights remain active. One optimizer step per rollout batch corresponds to $K = 1$, so there is no reuse-induced reward staleness. **[TODO: Record the exact configuration for gap smoothing, floors, and weight normalization from the integrated implementation.]**

Tail-aware supervision. TA-OPD (Huang & Wei, 2026) groups the omitted vocabulary into one tail event. For teacher support $S_i(c)$, write $p_{\mathrm{tail}} = 1 - \sum_{a \in S_i(c)} p_\theta(a \mid c)$ and $q_{\mathrm{tail}} =$

$1 - \sum_{a \in S_i(c)} q_i(a \mid c)$. Its position loss is

$$\ell_i^{\mathrm{TA}} = \sum_{a \in S_i(c)} p_\theta(a \mid c) \log \frac{p_\theta(a \mid c)}{q_i(a \mid c)} + p_{\mathrm{tail}} \log \frac{p_{\mathrm{tail}}}{q_{\mathrm{tail}}}. \tag{10}$$

Both runs use $k = 64$, the same response averaging and fixed task weights, and the same training budget as the core comparison. Only the allocation differs between TA-OPD + Uniform and TA-OPD + GPAS.

A.4 Step-local objective

Let θ_t be the current student and $\mathcal{R}_i^t$ its distribution over task-i prompts and responses at step t. The local objective differentiated by dense OPD is

$$F_t(\theta) = \sum_i w_i \, \mathbb{E}_{(x,y) \sim \mathcal{R}_i^t} \left[\frac{1}{|y|} \sum_{u=1}^{|y|} \ell_i^{(k)}(\theta; x, y_{<u}) \right]. \tag{11}$$

The rollout distribution is held fixed in this derivative. Thus the mean micro-batch update at θ_t is $\nabla F_t(\theta_t)$. The distribution is refreshed at the next step. Fixing w_i preserves the chosen task weighting through this sequence of local objectives.

A.5 Variance and the allocation rule

Condition on the current state and pre-step scaling D. In the independent micro-batch model, cross-task centered terms have zero expectation, giving

$$\mathbb{E}\|D(A - \mu)\|_2^2 = \sum_i w_i^2 \mathbb{E}\|D(\bar{g}_i - \mu_i)\|_2^2 = \sum_i \frac{w_i^2 e_i}{m_i}. \tag{12}$$

Equation 6 is the ordinary sample variance summed over preconditioned coordinates. Writing $a_i = w_i \sqrt{e_i}$, Cauchy–Schwarz gives

$$\left(\sum_i m_i \right) \left(\sum_i \frac{a_i^2}{m_i} \right) \geq \left(\sum_i a_i \right)^2. \tag{13}$$

Equality holds at $m_i \propto a_i$, yielding the continuous per-step rule. With unequal costs and zero fixed step time,

$$\left(\sum_i m_i \tau_i \right) \left(\sum_i \frac{a_i^2}{m_i} \right) \geq \left(\sum_i a_i \sqrt{\tau_i} \right)^2, \tag{14}$$

with equality at $m_i \propto a_i / \sqrt{\tau_i}$. Positive fixed time and integer bounds are handled by the finite search used in the cost-aware implementation.

A.6 Connection to a local update

For an L-smooth F_t and the frozen-preconditioner step $\theta' = \theta - \eta D A$, let $\mu = \nabla F_t(\theta)$. Smoothness and the bias–variance identity give

$$\mathbb{E} F_t(\theta') \leq F_t(\theta) - \eta \mu^\top D \mu + \frac{\eta^2 L}{2} \left(\|D\mu\|_2^2 + V(m) \right). \tag{15}$$

At the fixed state, the allocation-dependent term is $V(m)$. This motivates measuring noise in update coordinates. GPAS uses this local criterion with ordinary AdamW; momentum, clipping, and the new second-moment update remain part of that optimizer.

A.7 TIMING AND RELEASED TRACES

Task blocks record rollout, teacher-scoring, backward, and statistics time, together with response lengths and peak memory. The marginal per-micro-batch estimates form τ_i; synchronization, optimizer work, and other count-independent step operations form C. Their EMA estimates use decay 0.9. We compare predicted and observed step times and evaluate efficiency with actual elapsed time on both reserved GPUs. Evaluation compute is reported separately. If a deployment overlaps task blocks, its timing predictor should represent that execution schedule.

Checkpoints retain optimizer state, task stream positions, counts, and noise and time averages. The released traces include per-task held-out loss, prompt consumption, response length and truncation, micro-batch counts, noise estimates, and measured step time. **[TODO: Add the artifact location and populate the configuration and empirical figures from these records.]**